Database Management System (BCS-501)

AKTU B.Tech CSE, 3rd Year, Odd Semester - Revision Notebook Compiled for: Harsh Katiyar, PSIT Kanpur

---

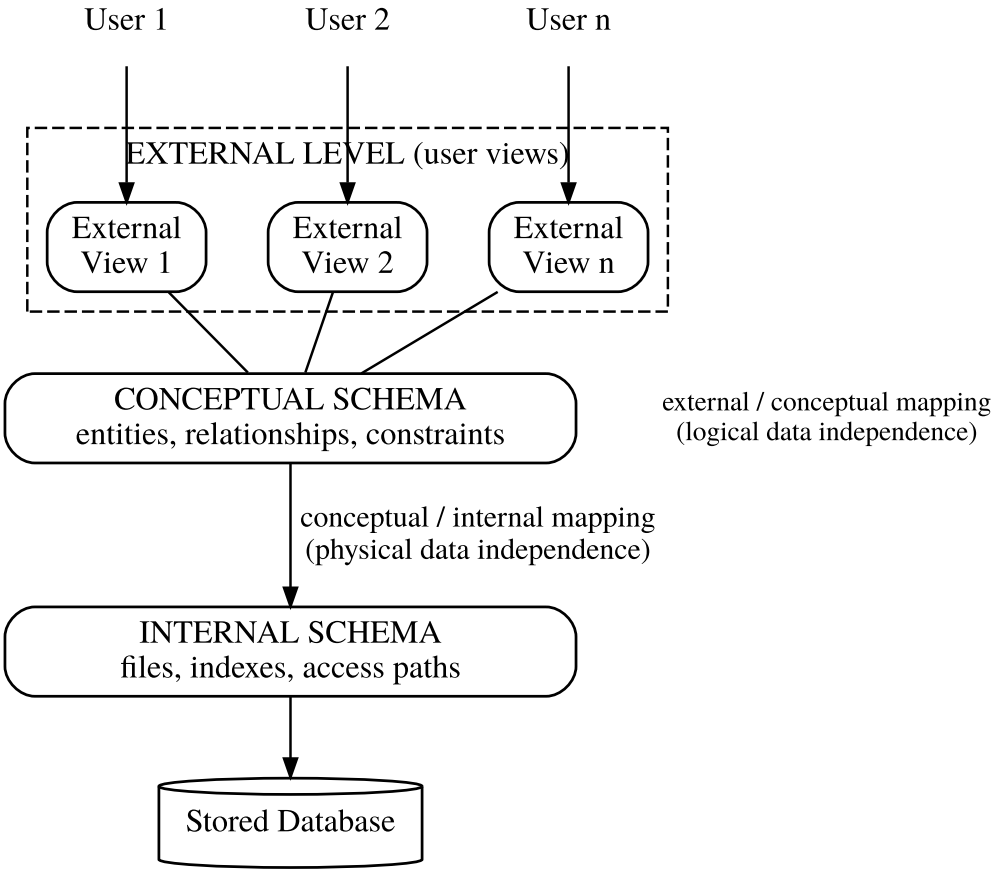
Unit 1: Architecture & ER Modeling

Q1. Explain the three-schema architecture (Internal, Conceptual, External) and how it achieves physical and logical data independence. (7 Marks)

1. Introduction / Definition

The **three-schema architecture** (ANSI/SPARC architecture) separates a database into three levels of abstraction: the **external**, **conceptual** and **internal** level. Its main goal is to separate the user's view of the data from the way it is physically stored. This separation gives us **data independence**.

2. Block Diagram



Three-schema (ANSI/SPARC) architecture

3. Core Explanation / Working

(a) Internal Level (Physical Level)

- Lowest level of abstraction. Describes **how** data is actually stored on disk.
- Covers storage structures, file organization, indexes, hashing, compression, record placement and access paths.
- Example: `STUDENT` records stored in a B+ tree file indexed on `Roll_No`.

(b) Conceptual Level (Logical Level)

- Describes **what** data is stored in the whole database and the relationships among the data.
- Hides physical storage details. Covers entities, attributes, data types, relationships, constraints and security.
- Designed and managed by the **DBA**.
- Example: `STUDENT(Roll_No, Name, DOB, Dept_ID)`, `DEPARTMENT(Dept_ID, DName)`.

(c) External Level (View Level)

- Highest level. Describes only the **part** of the database that a particular user group needs.
- Many external views can exist for one conceptual schema.
- Example: the Accounts section sees `(Roll_No, Name, Fee_Status)`. The Exam cell sees `(Roll_No, Name, Marks)`.

(d) Mappings

- **External/Conceptual mapping:** translates a request on a user view into a request on the conceptual schema.
- **Conceptual/Internal mapping:** translates conceptual records into stored records and access paths.

(e) Data Independence

- **Physical data independence:** the internal schema can change without changing the conceptual schema or application programs. Only the conceptual/internal mapping is updated.
  - Example: adding an index, moving to a new disk, or changing a file from heap to hashed organization.
- **Logical data independence:** the conceptual schema can change without changing external views or application programs. Only the external/conceptual mapping is updated.
  - Example: adding a new attribute `Email` to `STUDENT`, or splitting one table into two.
  - It is harder to achieve than physical independence, because applications depend directly on the logical structure.

4. Key Characteristics - Physical vs. Logical Data Independence

Basis	Physical Data Independence	Logical Data Independence
Change made at	Internal level	Conceptual level
Protected level	Conceptual schema	External schemas / views
Mapping updated	Conceptual / Internal	External / Conceptual
Example change	New index, new storage device	Add/remove attribute or table
Ease of achieving	Easy	Difficult
Concerned with	Storage efficiency	Structure of data

5. Advantages & Disadvantages

Advantages

- Gives data independence, which lowers application maintenance cost.
- Each user gets a customised view, which improves **security** by hiding unrelated data.
- The DBA can tune physical storage without affecting users.
- Supports many users with different needs on one shared database.

Disadvantages

- Mapping between levels adds **processing overhead** and can slow queries.
- More complex to design and to implement in the DBMS.
- Full logical independence is hard to achieve in practice.

6. Conclusion / Application

The three-schema architecture is the basis of all modern RDBMSs, such as Oracle, MySQL and PostgreSQL, where SQL **views** are the external level and **tablespaces/indexes** are the internal level.

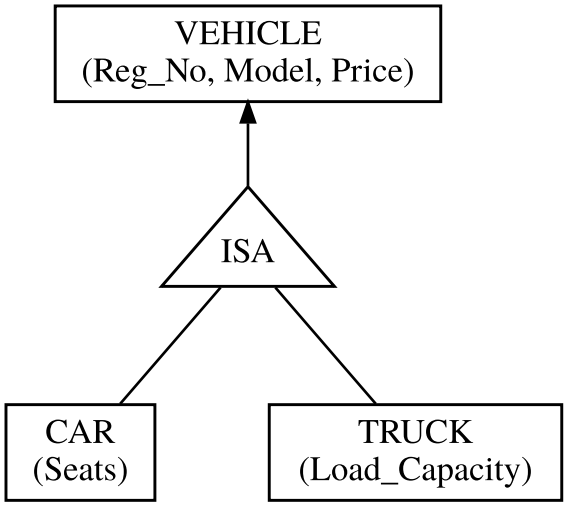
Q2. Differentiate between Generalization, Specialization and Aggregation using concrete examples. (7 Marks)

1. Introduction / Definition

Generalization, specialization and aggregation are **Enhanced ER (EER)** concepts that model **class hierarchies** and **relationships among relationships**. Generalization and specialization use the **IS-A** relationship (inheritance). Aggregation treats a relationship set as a higher-level entity.

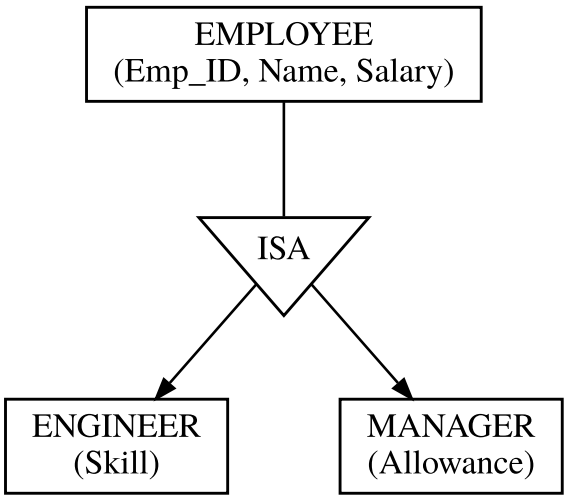
2. Diagrams

(a) Generalization (Bottom-Up)



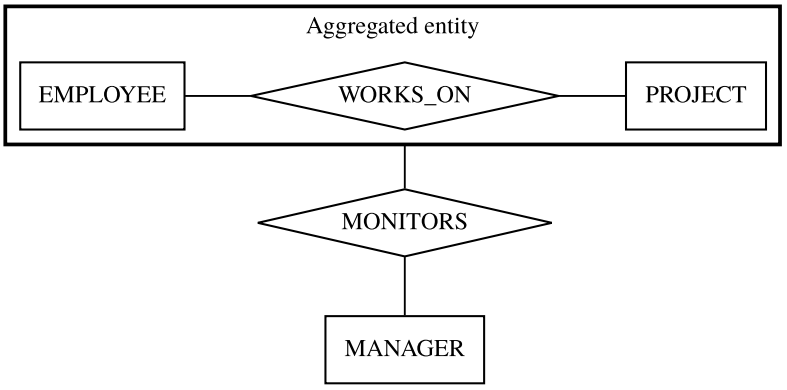
Generalization (bottom-up): common attributes move up to VEHICLE

(b) Specialization (Top-Down)



Specialization (top-down): EMPLOYEE split into sub-classes

(c) Aggregation



Aggregation: the relationship (EMPLOYEE works on PROJECT) is treated as one entity

3. Core Explanation / Working

(a) Generalization

- A **bottom-up** process: two or more lower-level entities with common attributes are combined into a **single higher-level entity**.
- Common attributes move to the super-class. Distinct attributes stay in the sub-classes.
- Example: `CAR (Reg_No, Model, Seats)` and `TRUCK (Reg_No, Model, Load_Capacity)` are generalized into `VEHICLE (Reg_No, Model)`.

(b) Specialization

- A **top-down** process: a higher-level entity is divided into **sub-classes** based on a distinguishing feature.
- Sub-classes **inherit** all attributes and relationships of the super-class and add their own.
- Example: `EMPLOYEE` is specialized into `ENGINEER (Skill)` and `MANAGER (Allowance)`.

(c) Constraints on Generalization/Specialization

- **Disjoint:** an entity belongs to at most one sub-class (an employee is either an Engineer or a Manager).

- **Overlapping:** an entity may belong to more than one sub-class (a person can be both a Student and an Employee).
- **Total participation:** every super-class entity must belong to some sub-class.
- **Partial participation:** some super-class entities may belong to no sub-class.

(d) Aggregation

- The ER model **cannot express relationships between relationships**.
- Aggregation is an **abstraction** in which a relationship set, together with its participating entity sets, is treated as **one higher-level entity**.
- Example: `EMPLOYEE --WORKS_ON-- PROJECT` is aggregated, and `MANAGER` then has a `MONITORS` relationship with this aggregate.
- Without aggregation we would need a ternary relationship, which causes **redundancy**.

4. Key Characteristics - Comparison Table

Basis	Generalization	Specialization	Aggregation
Approach	Bottom-up	Top-down	Abstraction of a relationship
Process	Combines entities into a super-class	Splits an entity into sub-classes	Treats a relationship as an entity
Relationship type	IS-A	IS-A	HAS-A / Part-of
Size of schema	Reduces	Increases	Avoids redundant ternary relationships
Inheritance	Sub-class attributes move up	Sub-classes inherit from super-class	No inheritance
Example	Car, Truck -> Vehicle	Employee -> Engineer, Manager	(Employee-Works_On-Project) monitored by Manager

5. Advantages & Disadvantages

Advantages

- Generalization removes **redundancy** of common attributes.
- Specialization models real-world **sub-types** precisely, with their own attributes.
- Aggregation can model **relationships among relationships** without redundant ternary sets.
- Together they make the ER diagram closer to object-oriented design.

Disadvantages

- They make the ER diagram more **complex** to read and draw.
- Mapping these hierarchies to tables needs extra design choices (one table per class, or one table for the whole hierarchy), which can cause **NULL values** or extra joins.

6. Conclusion / Application

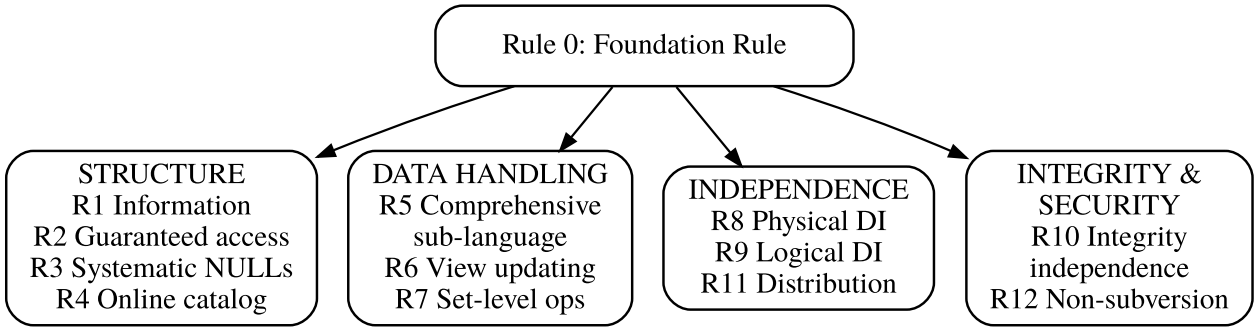
These EER concepts are used to design databases for **ERP, banking and university systems**, where entity hierarchies (Employee types, Account types) and monitored relationships (Project supervision) are common.

Q3. List and explain E.F. Codd's 12 rules that define a fully relational database system. (7 Marks)

1. Introduction / Definition

In 1985 **Dr. E.F. Codd**, the father of the relational model, proposed **13 rules (Rule 0 to Rule 12)**. A DBMS must satisfy them to be called a **fully relational DBMS**. They make sure the system manages data **only through its relational capabilities**.

2. Block Diagram - Classification of Codd's Rules



Codd's rules grouped by purpose

3. Core Explanation - The Rules

- **Rule 0 - Foundation Rule:** the system must manage databases **entirely** through its relational capabilities.
- **Rule 1 - Information Rule:** all information, including metadata, is represented **only as values in tables** (rows and columns).
- **Rule 2 - Guaranteed Access Rule:** every value must be reachable by the combination **Table name + Primary key value + Column name**, without using pointers.
- **Rule 3 - Systematic Treatment of NULL Values:** NULL must be handled consistently as **missing or inapplicable** information, distinct from zero or a blank string.
- **Rule 4 - Active Online Catalog (Data Dictionary):** the database description (metadata) is stored **in tables** and can be queried with the same language used for data.
- **Rule 5 - Comprehensive Data Sub-language Rule:** at least one language (like **SQL**) must support data definition, data manipulation, views, integrity constraints, authorization and transaction control.
- **Rule 6 - View Updating Rule:** every view that is **theoretically updatable** must be updatable by the system.
- **Rule 7 - High-Level Insert, Update, Delete:** insert, update and delete must work on **sets of rows** at a time, not only on a single row.
- **Rule 8 - Physical Data Independence:** changes to storage or access methods must **not affect** application programs.
- **Rule 9 - Logical Data Independence:** changes to the logical structure (such as adding a column or splitting a table) must **not affect** applications or views.
- **Rule 10 - Integrity Independence:** integrity constraints are defined in the sub-language and **stored in the catalog**, not in application programs.
- **Rule 11 - Distribution Independence:** users must not be aware whether data is **distributed** across sites. Applications keep working when data is distributed.
- **Rule 12 - Non-Subversion Rule:** if a low-level (record-at-a-time) interface exists, it must **not bypass** the integrity and security rules of the relational language.

4. Key Characteristics - Rule Summary Table

Rule	Name	Key Idea
0	Foundation	Fully relational management
1	Information	Everything stored in tables
2	Guaranteed Access	Table + PK + Column
3	NULL Treatment	Uniform handling of missing values
4	Online Catalog	Metadata stored as tables
5	Data Sub-language	One complete language (SQL)
6	View Updating	Updatable views are updatable
7	Set-level Operations	Insert/Update/Delete on sets
8	Physical DI	Storage changes are invisible
9	Logical DI	Schema changes are invisible
10	Integrity Independence	Constraints stored in catalog
11	Distribution Independence	Location transparency
12	Non-subversion	No bypassing of constraints

Note: no commercial RDBMS (Oracle, MySQL, SQL Server) satisfies **all 12 rules fully**. Most satisfy about 8-10 rules. Rules 6, 9 and 11 are the ones most often only partly met.

5. Advantages & Disadvantages

Advantages (of following Codd's rules)

- They give a **standard benchmark** for judging whether a DBMS is truly relational.
- They ensure **data independence, integrity and security**.
- They promote set-based processing, which makes queries **simpler and more powerful**.

Disadvantages

- Strict compliance is **hard to implement**, and no commercial product achieves it fully.
- Some rules, like full view updatability, are **theoretically complex** and can reduce performance.

6. Conclusion / Application

Codd's rules are the **yardstick used by vendors and academics** to rate RDBMS products such as Oracle, DB2 and PostgreSQL on how relational they are.

Q4. Short Definitions (2 Marks Each)

(a) Data Dictionary

- A **data dictionary** is a centralized repository of **metadata** ("data about data").
- It stores table names, attribute names, data types, constraints, indexes, views, users and access rights.
- An **active** data dictionary is updated automatically by the DBMS. A **passive** one is maintained manually.
- Example: `INFORMATION_SCHEMA` in MySQL.

(b) Database Schema vs. Instance

Schema	Instance
The **logical design / structure** of the database	The **actual data** stored at a particular moment
Defined once using DDL, rarely changes	Changes frequently with every insert/update/delete
Like a variable's type declaration	Like a variable's value
Example: `STUDENT(Roll, Name, Age)`	Example: `(101, 'Harsh', 21)`

(c) Responsibilities of a DBA (Database Administrator)

- **Schema definition:** creates the conceptual schema using DDL.
- **Storage structure and access method definition:** chooses file organization and indexes.
- **Granting authorization:** controls user access and privileges (GRANT/REVOKE).
- **Backup and recovery:** takes periodic backups and restores after failures.
- **Integrity constraint specification:** defines rules to keep data consistent.
- **Performance tuning and monitoring:** monitors performance and tunes queries.

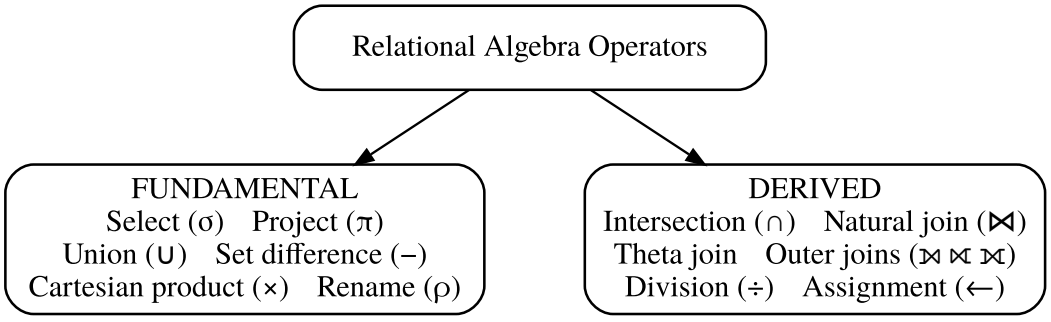
Unit 2: Relational Algebra & Calculus

Q1. Explain fundamental and derived relational algebra operators, with focus on Natural Join, Outer Joins and the Division operator. (7 Marks)

1. Introduction / Definition

Relational algebra is a **procedural** query language. It takes one or more relations as input and produces a new relation as output. Its operators are divided into **fundamental** (basic) operators and **derived** operators, which can be expressed using the basic ones.

2. Block Diagram - Classification of Operators



Classification of relational algebra operators

3. Core Explanation / Working

(a) Fundamental Operators

- **Select (σ):** chooses **rows** that satisfy a predicate. `σ Dept='CSE' (STUDENT)`
- **Project (π):** chooses **columns** and removes duplicates. `π Name, Dept (STUDENT)`

- **Union (∪):** all tuples in R or S. R and S must be **union compatible** (same degree and domains).
- **Set Difference (−):** tuples in R but not in S.
- **Cartesian Product (×):** every tuple of R combined with every tuple of S. Result size is |R| x |S|.
- **Rename (ρ):** renames a relation or its attributes. `ρ S(STUDENT)`

(b) Derived Operators

- **Intersection:** `R ∩ S = R − (R − S)`
- **Theta Join:** `R ⋈θ S = σθ (R × S)`

Sample relations used below

EMP

EID	Name	DID
1	Amit	10
2	Riya	20
3	Karan	40

DEPT

DID	DName
10	CSE
20	IT
30	ECE

(c) Natural Join (EMP ⋈ DEPT)

- Joins on **all common attributes** (here `DID`) with an equality condition and keeps **one copy** of each common column.
- Formula: `R ⋈ S = π (R ∪ S attrs) ( σ R.A = S.A (R × S) )`

EID	Name	DID	DName
1	Amit	10	CSE
2	Riya	20	IT

(d) Outer Joins

- They keep **unmatched tuples** as well, filling the missing values with **NULL**. This avoids the information loss of a natural join.
- **Left Outer Join (EMP ⟕ DEPT):** all EMP tuples are kept.
- **Right Outer Join (EMP ⟖ DEPT):** all DEPT tuples are kept.
- **Full Outer Join (EMP ⟗ DEPT):** all tuples of both relations are kept.

Left outer join (EMP ⟕ DEPT)

EID	Name	DID	DName
1	Amit	10	CSE
2	Riya	20	IT
3	Karan	40	NULL

Right outer join (EMP ⋈ DEPT)

EID	Name	DID	DName
1	Amit	10	CSE
2	Riya	20	IT
NULL	NULL	30	ECE

Full outer join (EMP ⋈ DEPT) = all four rows above: Amit, Riya, Karan (DName NULL) and ECE (EID, Name NULL).

(e) Division Operator (R ÷ S)

- Used for **"for all"** queries, for example "find students who are enrolled in **all** courses".
- If R(A, B) and S(B), then R ÷ S gives every A value that is paired with **every** B value in S.
- Formula: `R ÷ S = π A (R) − π A ( (π A (R) × S) − R )`

ENROLLED (R): Student	Course
A	C1
A	C2
B	C1
C	C1
C	C2
C	C3

COURSE (S): Course
C1
C2

R ÷ S: Student
A
C

- *Step-by-step:*
  1. `π Student (R)` = {A, B, C}
  2. `π Student (R) × S` = {AC1, AC2, BC1, BC2, CC1, CC2}
  3. Subtract R: {BC2}. These are the missing pairs.
  4. `π Student` of step 3 = {B}
  5. {A, B, C} − {B} = **{A, C}**

4. Key Characteristics - Join Comparison

Join Type	Condition	Unmatched Tuples	Duplicate Columns
Theta Join	Any condition (<, >, =, etc.)	Discarded	Kept
Equi Join	Only equality (=)	Discarded	Kept
Natural Join	Equality on common attributes	Discarded	Removed
Left Outer	Equality	Left side kept (NULL padding)	Removed
Right Outer	Equality	Right side kept (NULL padding)	Removed
Full Outer	Equality	Both sides kept	Removed

5. Advantages & Disadvantages

Advantages

- It is the **formal foundation** of SQL and of query optimization.
- Operators are **closed**: the output is always a relation, so operations can be **nested**.
- Derived operators like join and division make complex queries **shorter**.

Disadvantages

- It is **procedural**: the user must specify *how* to get the result.
- It lacks aggregation, sorting and recursion in its basic form.
- Cartesian products and joins on large tables can be very **costly**.

6. Conclusion / Application

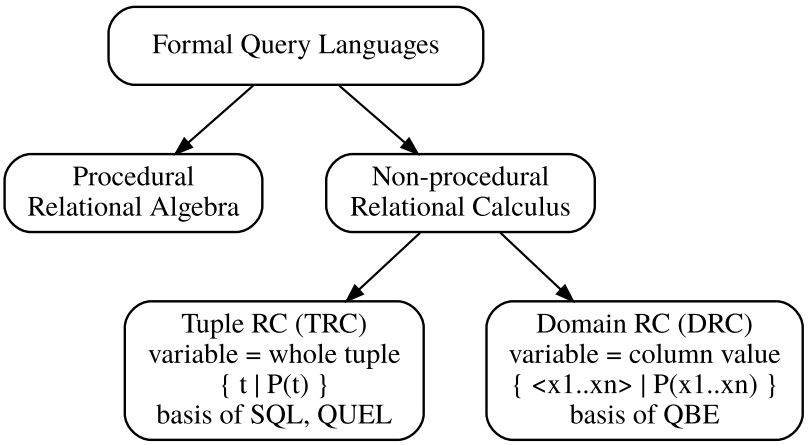
Relational algebra is used **internally by the query optimizer** of every RDBMS to turn SQL into efficient execution plans, for example by pushing selections before joins.

Q2. Differentiate between Tuple Relational Calculus (TRC) and Domain Relational Calculus (DRC) with syntax. (7 Marks)

1. Introduction / Definition

Relational calculus is a **non-procedural (declarative)** query language based on first-order predicate logic. The user states *what* data is required, not *how* to get it. It has two forms: **Tuple Relational Calculus (TRC)**, where variables range over **tuples**, and **Domain Relational Calculus (DRC)**, where variables range over **attribute values (domains)**.

2. Block Diagram



Formal query languages

3. Core Explanation / Working

Sample relation: `STUDENT(Roll, Name, Age, Dept)`

(a) Tuple Relational Calculus (TRC)

- **General form:** `{ t | P(t) }`, read as "the set of all tuples t for which predicate P is true".
- `t.A` means the value of attribute A in tuple t.
- Uses ∈, comparison operators (`<, >, =, ≠, ≤, ≥`), connectives (`∧, ∨, ¬, ⇒`) and quantifiers (∃ there exists, ∀ for all).
- *Example 1:* names of students older than 20: `{ t.Name | t ∈ STUDENT ∧ t.Age > 20 }`
- *Example 2 (with a join):* names of students in the 'CSE' department, using `DEPT(Dept, HOD)`: `{ t.Name | t ∈ STUDENT ∧ ∃ d ∈ DEPT ( d.Dept = t.Dept ∧ d.Dept = 'CSE' ) }`

(b) Domain Relational Calculus (DRC)

- **General form:** `{ <x1, x2, ..., xn> | P(x1, x2, ..., xn) }`
- Each `xi` is a **domain variable** that stands for a single attribute value.
- `<r, n, a, d> ∈ STUDENT` means a tuple with those values exists in STUDENT.
- *Example 1:* names of students older than 20: `{ <n> | ∃ r, a, d ( <r, n, a, d> ∈ STUDENT ∧ a > 20 ) }`
- *Example 2:* roll number and name of CSE students: `{ <r, n> | ∃ a ( <r, n, a, 'CSE'> ∈ STUDENT ) }`

(c) Safety of Expressions

- An expression like `{ t | ¬(t ∈ STUDENT) }` is **unsafe** because it produces an infinite result.
- A **safe** expression only returns values from the **domain of the expression**, meaning values that appear in the relations or constants used in the query.
- Safe TRC, safe DRC and relational algebra have **equal expressive power**. This property is called **relational completeness**.

4. Key Characteristics - TRC vs. DRC

Basis	Tuple Relational Calculus (TRC)	Domain Relational Calculus (DRC)
Variable represents	An entire **tuple** (row)	A single **attribute value** (column)
General syntax	`{ t | P(t) }`	`{ <x1,...,xn> | P(x1,...,xn) }`
Attribute access	Dot notation `t.Name`	Positional `<r, n, a, d>`
Membership test	`t ∈ R`	`<x1,...,xn> ∈ R`
Query language based on it	SQL, QUEL (Ingres)	QBE (Query-By-Example)
Readability	Easier for multi-attribute relations	Lengthy, since all columns must be listed
Expressive power	Same as relational algebra (safe expressions)	Same as relational algebra (safe expressions)

5. Advantages & Disadvantages

Advantages

- It is **declarative**: users state *what* they want, which is close to natural logic.
- It allows **query optimization freedom**, because the DBMS chooses the execution strategy.
- It is **relationally complete**, with the same power as relational algebra.

Disadvantages

- Unsafe expressions can give **infinite results**, so safety checks are needed.
- Quantifiers (`∀`, `∃`) make complex queries **hard to write and understand** for beginners.
- Like basic algebra, it does not support aggregation directly.

6. Conclusion / Application

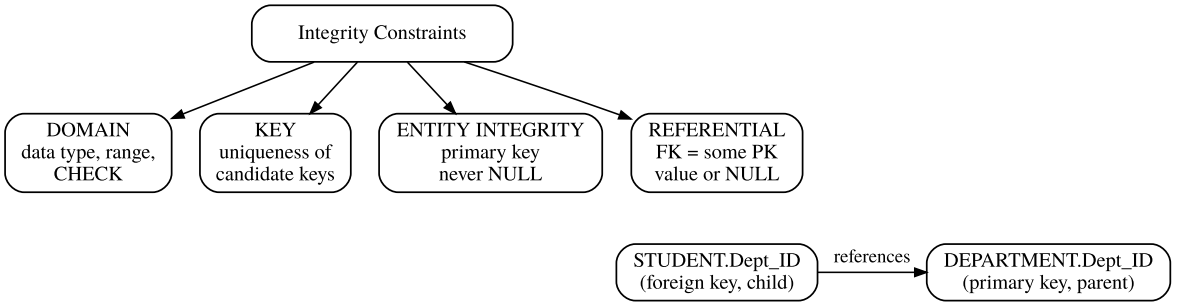
TRC is the theoretical basis of **SQL's** `SELECT ... FROM ... WHERE` and DRC is the basis of **QBE** interfaces like the MS Access query designer.

Q3. Explain Domain constraints, Entity Integrity and Referential Integrity with table examples. (7 Marks)

1. Introduction / Definition

Integrity constraints are rules applied to a database to keep its data **accurate, valid and consistent**. They are defined in the schema (using DDL) and the DBMS **enforces them automatically** on every insert, update and delete.

2. Block Diagram



Types of integrity constraints, and the foreign-key link

3. Core Explanation / Working

(a) Domain Constraint

- Each attribute value must be an **atomic** value from its defined **domain** (data type, length, range, format).
- Defined using data types, `NOT NULL`, `DEFAULT` and `CHECK`.
- SQL: `Age INT CHECK (Age BETWEEN 17 AND 30)`

STUDENT

Roll	Name	Age	Status
101	Harsh	21	Valid
102	Neha	‘abc’	**Violation:** Age must be INT
103	Ravi	45	**Violation:** CHECK (17 to 30) fails

(b) Key Constraint

- Every candidate key must be **unique**. No two tuples may have the same key value.

(c) Entity Integrity Constraint

- **No primary key attribute may be NULL.** The primary key identifies each tuple uniquely, and NULL cannot identify anything.
- SQL: `Roll INT PRIMARY KEY` (this implies `NOT NULL` and `UNIQUE`).

STUDENT

Roll	Name	Status
101	Harsh	Valid
NULL	Aman	**Violation:** PK cannot be NULL
101	Priya	**Violation:** duplicate PK (key constraint)

(d) Referential Integrity Constraint

- A **foreign key (FK)** value in the referencing (child) table must either **match an existing primary key** value in the referenced (parent) table or be **NULL**.
- SQL: `Dept_ID INT REFERENCES DEPARTMENT(Dept_ID)`

DEPARTMENT (parent)

Dept_ID	DName
10	CSE
20	IT

STUDENT (child)

Roll	Name	Dept_ID	Status
101	Harsh	10	Valid
102	Neha	20	Valid
103	Ravi	NULL	Valid (not assigned yet)
104	Mohit	50	**Violation:** 50 is not in DEPARTMENT

(e) How operations can violate constraints

- **Insert:** can violate domain, key, entity and referential integrity (child insert with a non-existent FK).
- **Delete:** can violate referential integrity only (deleting a parent row that child rows still reference).
- **Update:** can violate all of them.
- *Actions on violation:* `RESTRICT/NO ACTION` (reject), `CASCADE` (propagate), `SET NULL`, `SET DEFAULT`.

4. Key Characteristics - Comparison

Basis	Domain Constraint	Entity Integrity	Referential Integrity
Applies to	Every attribute	Primary key attributes	Foreign key attributes
Rule	Value from valid domain	PK ≠ NULL	FK = some PK value or NULL
Scope	Single attribute	Single relation	Two relations
SQL syntax	Data type, CHECK, NOT NULL	PRIMARY KEY	FOREIGN KEY … REFERENCES
Violated by	Insert, Update	Insert, Update	Insert (child), Delete (parent), Update

5. Advantages & Disadvantages

Advantages

- They keep the database **accurate and consistent** by preventing invalid data.
- Enforcement is **centralised** in the DBMS, so each application does not need validation code.
- Referential integrity prevents **orphan records** (dangling references).

Disadvantages

- Checking constraints on every operation adds **performance overhead**, especially on bulk loads.
- `CASCADE` actions can **unintentionally delete** large amounts of related data.

6. Conclusion / Application

Integrity constraints are used in every **banking, e-commerce and university ERP** system. For example, an order row can never refer to a non-existent customer.

Q4. Short Definitions (2 Marks Each)

(a) Cartesian Product (×)

- A binary operation that combines **every tuple of R with every tuple of S**.
- If R has m tuples and n1 attributes, and S has p tuples and n2 attributes, then R × S has **m x p tuples** and **n1 + n2 attributes**.
- Example: R with 3 rows × S with 4 rows = 12 rows.

(b) Theta Join (⋈θ)

- A join that combines tuples from two relations that satisfy a **general condition θ**, which can use any of `=, <, >, ≤, ≥, ≠`.
- `R ⋈θ S = σθ (R × S)`
- Example: `EMP ⋈ (EMP.Salary > MGR.Salary) MGR`. When θ uses only =, it is called an **Equi Join**.

(c) Foreign Key and Cascading Deletes

- When a foreign key is declared with **`ON DELETE CASCADE`**, deleting a parent (referenced) row **automatically deletes all child rows** that refer to it. This keeps referential integrity without leaving orphan rows.
- SQL: `FOREIGN KEY (Dept_ID) REFERENCES DEPARTMENT(Dept_ID) ON DELETE CASCADE`
- Example: deleting `Dept_ID = 10` from DEPARTMENT also deletes every STUDENT row with `Dept_ID = 10`.

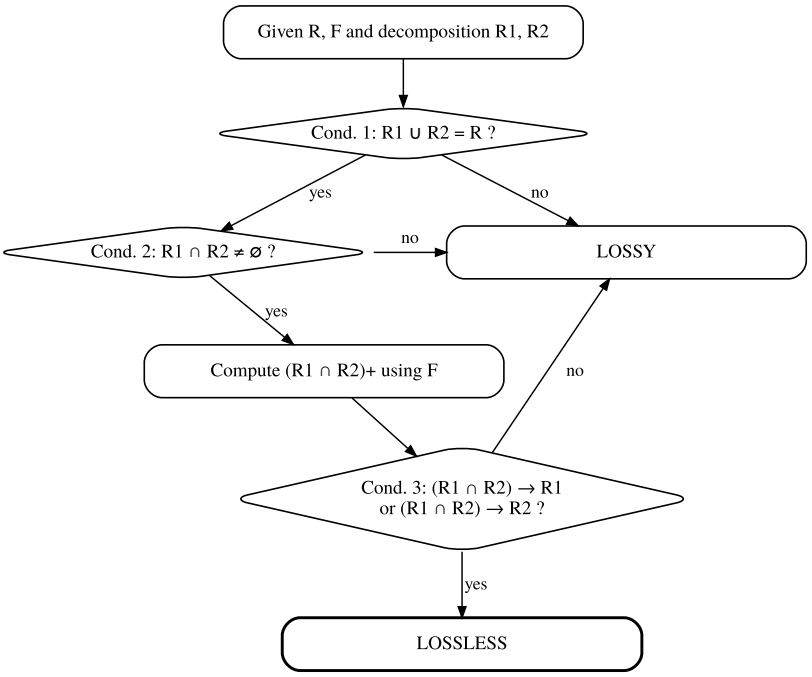
Unit 3: Database Design & Normalization

Q1. Prove whether a given database decomposition is lossless or lossy, and explain dependency preservation. (7 Marks)

1. Introduction / Definition

A decomposition of relation R into R1 and R2 is **lossless (non-additive) join** if joining R1 and R2 gives back **exactly** the original R, with no extra (spurious) tuples: `R1 ⋈ R2 = R`. If the join produces spurious tuples, the decomposition is **lossy**. A decomposition is **dependency preserving** if every functional dependency of R can be checked using the decomposed relations alone.

2. Flowchart - Lossless Join Test (Binary Decomposition)



Lossless-join test for a binary decomposition

3. Core Explanation / Working

(a) Conditions for a lossless join (R into R1, R2)

1. `Attr(R1) ∪ Attr(R2) = Attr(R)`
2. `Attr(R1) ∩ Attr(R2) ≠ ∅`
3. `R1 ∩ R2 → R1` **or** `R1 ∩ R2 → R2` must hold in F+. In other words, the common attribute(s) must be a **superkey** of at least one of the two relations.

(b) Solved Example 1 - LOSSLESS

`R(A, B, C, D, E), F = { A → BC, CD → E, B → D, E → A }` Decomposition: `R1(A, B, C)` and `R2(A, D, E)`

- Cond 1: {A,B,C} ∪ {A,D,E} = {A,B,C,D,E} = R. **Satisfied.**
- Cond 2: R1 ∩ R2 = {A} ≠ ∅. **Satisfied.**
- Cond 3: compute A+.
  - Start: A+ = {A}
  - A → BC gives {A, B, C}
  - B → D gives {A, B, C, D}
  - CD → E gives {A, B, C, D, E}
  - So A+ = ABCDE, and A → ABC (= R1) holds. **Satisfied.**
- **Result: the decomposition is LOSSLESS.**

(c) Solved Example 2 - LOSSY (shown with data)

`R(A, B, C), F = { A → B }`, decomposed into `R1(A, B)` and `R2(B, C)`.

- R1 ∩ R2 = {B}, and B+ = {B}. B is a key of neither R1 nor R2, so the decomposition is **LOSSY**.

R: A	B	C
1	x	p
2	x	q

R1: A	B
1	x
2	x

R2: B	C
x	p
x	q

R1 ⋈ R2: A	B	C	Note
1	x	p	original
1	x	q	**spurious**
2	x	p	**spurious**
2	x	q	original

The join gives 4 tuples instead of 2. The extra tuples make information **ambiguous**, which is why the decomposition is called “lossy”.

(d) Decomposition into more than two relations (successive joins)

- `R(A,B,C,D)`, `F = {A → B, B → C, C → D}`, decomposition `R1(A,B), R2(B,C), R3(C,D)`.
- R1 ⋈ R2: common attribute B, and B → C, so B is a key of R2. Lossless, giving `R12(A,B,C)`.
- R12 ⋈ R3: common attribute C, and C → D, so C is a key of R3. Lossless, giving `R(A,B,C,D)`.
- **Result: LOSSLESS.** For harder cases, use the **Chase (Tableau) matrix method**.

(e) Dependency Preservation

- Let Fi be the set of FDs in F+ that contain **only attributes of Ri** (the projection of F onto Ri).
- The decomposition is **dependency preserving** if `(F1 ∪ F2 ∪ ... ∪ Fn)+ = F+`.
- *Checking Example 1:*
  - F1 on R1(A,B,C): `A → BC` and `BC → A`. BC → A holds because BC+ = BCDEA.
  - F2 on R2(A,D,E): `A → DE` and `E → A`.
  - Is `B → D` implied by F1 `∪` F2? B+ = {B}, which does not contain D. **Not preserved.**
  - Is `CD → E` implied? CD+ = {C, D}, which does not contain E. **Not preserved.**
  - **Conclusion:** the decomposition in Example 1 is **lossless but NOT dependency preserving**.

4. Key Characteristics - Lossless vs. Lossy / Dependency Preservation

Basis	Lossless Join	Lossy Join	Dependency Preservation
Join result	Exactly the original R	R plus spurious tuples	Not about the join
Test	Common attr is a key of R1 or R2	Test fails	(∪ Fi)+ = F+
Necessity	**Mandatory**	Never acceptable	Desirable (may be given up for BCNF)
Guaranteed by	3NF and BCNF algorithms	-	3NF synthesis (not always BCNF)

5. Advantages & Disadvantages

Advantages

- A lossless decomposition **removes redundancy** without losing information.
- Dependency preservation lets constraints be **checked locally** on single tables without costly joins.
- Together they guarantee a **correct normalization** (3NF gives both properties).

Disadvantages

- A lossless decomposition still needs **joins** at query time, which costs performance.
- **BCNF may not preserve dependencies**, so a designer sometimes has to accept 3NF.

6. Conclusion / Application

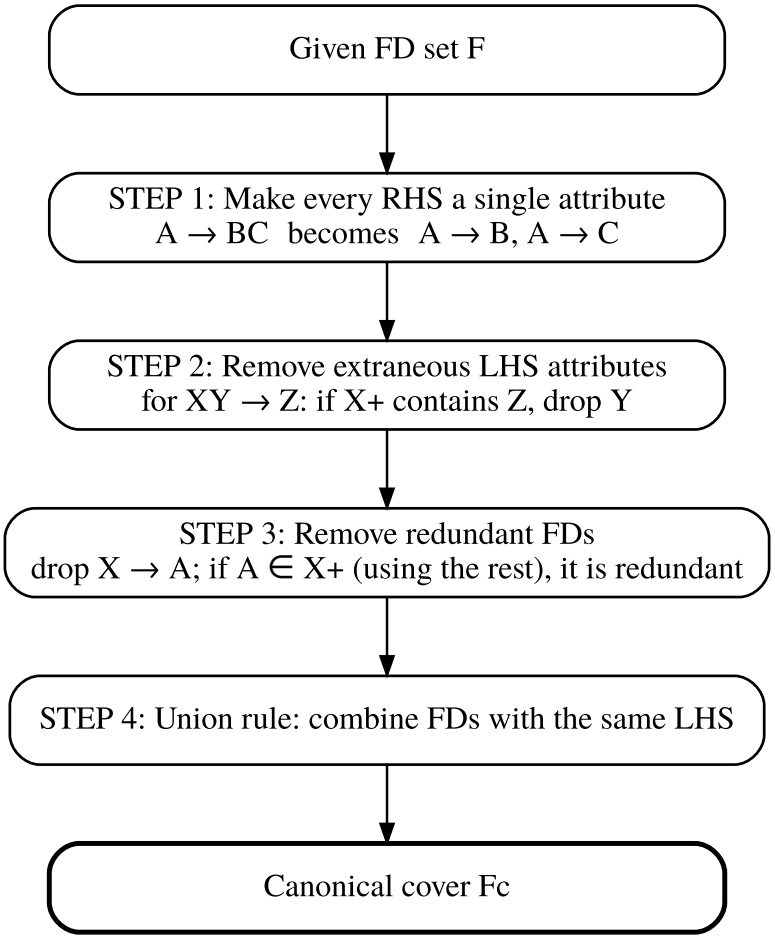
The lossless-join test is used in **schema normalization tools** and by database designers to make sure that splitting tables (for example in an ERP schema) never corrupts the stored information.

Q2. Calculate the minimal cover (canonical cover) for a given set of functional dependencies. (7 Marks)

1. Introduction / Definition

A **canonical (minimal) cover Fc** of an FD set F is the **smallest equivalent set** of FDs such that: (i) every FD has a **single attribute on the RHS**, (ii) no FD has an **extraneous attribute** on its LHS, and (iii) no FD is **redundant**. Fc has the same closure as F (`Fc+ = F+`).

2. Flowchart - Algorithm



Algorithm for finding the canonical cover

3. Core Explanation / Working - Solved Example 1

Given: `F = { A → B, C → B, D → ABC, AC → D }`

Step 1: Make every RHS a single attribute

$$F_1 = \{A \to B,\ \ C \to B,\ \ D \to A,\ \ D \to B,\ \ D \to C,\ \ AC \to D\}$$

Step 2: Remove extraneous attributes from LHS (only `AC → D` has a composite LHS)

- Is A extraneous? Check C+ = {C, B}. D is not in it, so **A is needed**.
- Is C extraneous? Check A+ = {A, B}. D is not in it, so **C is needed**.
- `AC → D` stays as it is.

Step 3: Remove redundant FDs (for each FD, compute the LHS closure without that FD)

FD tested	Closure using the remaining FDs	RHS found?	Decision
A → B	A+ = {A}	No	Keep
C → B	C+ = {C}	No	Keep
D → A	D+ = {D, B, C}	No	Keep
D → B	D+ = {D, A, C, B} (via D→A, A→B)	**Yes**	**Remove**
D → C	D+ = {D, A, B}	No	Keep
AC → D	AC+ = {A, C, B}	No	Keep

Step 4: Combine FDs with the same LHS

- `D → A` and `D → C` combine to give `D → AC`.

Answer:

$$F_c = \{A \to B,\ \ C \to B,\ \ D \to AC,\ \ AC \to D\}$$

Solved Example 2 (Korth)

Given: `F = { A → BC, B → C, A → B, AB → C }`

- *Step 1:* `{ A → B, A → C, B → C, A → B, AB → C }`. Remove the duplicate to get `{ A → B, A → C, B → C, AB → C }`.
- *Step 2:* in `AB → C`, A+ = {A, B, C} already contains C, so **B is extraneous**. `AB → C` becomes `A → C`, which is a duplicate.
- *Step 3:* `A → C` is redundant, because A → B and B → C give A → C by transitivity. Remove it.
- **Answer: `Fc = { A → B, B → C }`**

4. Key Characteristics - F vs. Fc

Property	Original F	Canonical Cover Fc
RHS	May be composite	Single attribute (combined only at the end)
Extraneous attributes	May exist	None
Redundant FDs	May exist	None
Closure	F+	Fc+ = F+ (equivalent)
Uniqueness	-	**Not unique**, since several minimal covers may exist

5. Advantages & Disadvantages

Advantages

- It **reduces the cost of checking** FDs on every database update.
- It is the **input to the 3NF synthesis algorithm**, which gives a lossless and dependency-preserving design.
- It removes redundancy in the constraint set, making the design **simpler to analyse**.

Disadvantages

- Computing closures repeatedly is **time-consuming by hand** for large FD sets.
- The result is **not unique**: a different processing order may give a different (but equivalent) cover.

6. Conclusion / Application

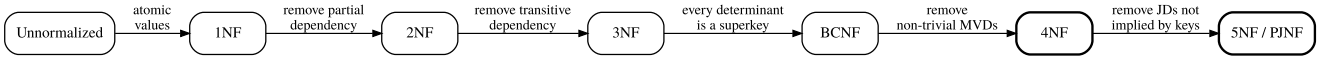
The canonical cover is used by **database design tools** and in the **3NF decomposition algorithm** to produce optimal table structures with the fewest constraints to enforce.

Q3. Explain Multivalued Dependency (MVD) leading to 4NF, followed by Join Dependency leading to 5NF. (7 Marks)

1. Introduction / Definition

A **Multivalued Dependency (MVD)** `X ↠ Y` exists when, for one value of X, there is a **set of Y values that is independent** of the other attributes. **4NF** removes non-trivial MVDs. A **Join Dependency (JD)** exists when a relation can be rebuilt losslessly **only by joining three or more** of its projections. **5NF (PJNF)** removes such join dependencies.

2. Block Diagram - Normal Form Hierarchy



Normal forms: each step removes one kind of dependency

3. Core Explanation / Working

(a) Multivalued Dependency (MVD)

- **Definition:** in R, `X ↠ Y` holds if whenever two tuples t1 and t2 agree on X, there exist tuples t3 and t4 such that:
  - t3[X] = t4[X] = t1[X] = t2[X]
  - t3[Y] = t1[Y] and t3[R−X−Y] = t2[R−X−Y]
  - t4[Y] = t2[Y] and t4[R−X−Y] = t1[R−X−Y]
- **Trivial MVD:** Y ⊆ X, or X ∪ Y = R.
- MVDs always occur in pairs: if `X ↠ Y` then `X ↠ (R − X − Y)` (the complementation rule).
- Every FD is also an MVD (if X → Y then X →→ Y).

(b) Example - Relation with an MVD (in BCNF, but not 4NF)

A student takes several courses and has several hobbies, and the two are **independent** of each other.

STUDENT_INFO (key = {SID, Course, Hobby}; MVDs: SID →→ Course and SID →→ Hobby)

SID	Course	Hobby
S1	DBMS	Cricket
S1	DBMS	Music
S1	OS	Cricket
S1	OS	Music

- Problem: adding one new hobby for S1 requires **2 new rows** (one per course). This is redundancy and an update anomaly.

(c) Fourth Normal Form (4NF)

- **Definition:** R is in 4NF if it is in **BCNF** and, for every **non-trivial MVD** `X ↠ Y`, **X is a superkey**.
- In the example, SID is not a superkey, so the relation is **not in 4NF**.
- **Decomposition:** for X →→ Y, split into R1(X ∪ Y) and R2(R − Y).

STUDENT_COURSE

SID	Course
S1	DBMS
S1	OS

STUDENT_HOBBY

SID	Hobby
S1	Cricket
S1	Music

Both tables are now in 4NF (only trivial MVDs remain), and the decomposition is lossless.

(d) Join Dependency (JD)

- **Definition:** R satisfies the join dependency `*(R1, R2, ..., Rn)` if R is always equal to the natural join of its projections: `R = π R1 (R) ⋈ π R2 (R) ⋈ ... ⋈ π Rn (R)`.
- An MVD is a special case of a JD with n = 2.
- A JD is **trivial** if one of the Ri equals R.

(e) Example - Relation with a JD (in 4NF, but not 5NF)

Rule (cyclic constraint): if a Supplier supplies Part P, the Supplier supplies to Project J, and Project J uses Part P, then the Supplier supplies P to J.

SPJ (all attributes form the key; no MVD exists)

Supplier	Part	Project
S1	P1	J2
S1	P2	J1
S2	P1	J1
S1	P1	J1

- Projections: `SP = {S1P1, S1P2, S2P1}`, `PJ = {P1J2, P2J1, P1J1}`, `JS = {J2S1, J1S1, J1S2}`.
- `SP ⋈ PJ` = {S1P1J2, S1P1J1, S1P2J1, **S2P1J2**, S2P1J1}. The tuple S2P1J2 is **spurious**, so 2 projections are not enough.
- `(SP ⋈ PJ) ⋈ JS` removes S2P1J2, because (J2, S2) is not in JS. The result is exactly SPJ.
- So `*(SP, PJ, JS)` holds and is **not implied by the candidate key**.

(f) Fifth Normal Form (5NF / Project-Join Normal Form)

- **Definition:** R is in 5NF if it is in 4NF and **every non-trivial join dependency is implied by the candidate keys**.
- **Decomposition:** replace SPJ with the three tables `SP(Supplier, Part)`, `PJ(Part, Project)` and `JS(Project, Supplier)`. Each one is in 5NF.

4. Key Characteristics - 4NF vs. 5NF

Basis	4NF	5NF (PJNF)
Dependency removed	Multivalued dependency (MVD)	Join dependency (JD)
Pre-condition	Must be in BCNF	Must be in 4NF
Condition	For every non-trivial X $\rightarrow\rightarrow$ Y, X is a superkey	Every JD is implied by candidate keys
Decomposed into	2 relations	3 or more relations
Typical cause	Independent multi-valued facts	Cyclic constraint among 3+ entities
Example	SID, Course, Hobby	Supplier, Part, Project

5. Advantages & Disadvantages

Advantages

- They **remove the redundancy** caused by independent multi-valued facts, which BCNF cannot remove.
- They eliminate **insertion, deletion and update anomalies** completely.
- 5NF is the **ultimate normal form** for project-join decomposition: the data cannot be decomposed any further without loss.

Disadvantages

- More tables means **more joins** and slower query performance.
- JDs are **hard to identify** in real schemas, so 5NF is rarely applied in practice.

6. Conclusion / Application

4NF and 5NF are applied in **data-warehouse staging and complex supply-chain systems**, where independent multi-valued facts (Supplier-Part-Project) would otherwise create huge redundancy.

Q4. Short Definitions (2 Marks Each)

(a) Update / Insertion Anomalies

- **Insertion anomaly:** a fact cannot be inserted without inserting other, unrelated facts. Example: a new department cannot be added until at least one student joins it, if both are stored in one table.
- **Update anomaly:** a change to one fact must be repeated in many rows. If one row is missed, the data becomes **inconsistent**. Example: a department's HOD name stored in every student row.
- (**Deletion anomaly:** deleting one fact unintentionally deletes another, for example deleting the last student also deletes the department details.)

(b) Trivial vs. Non-Trivial Functional Dependency

Trivial FD	Non-Trivial FD
`X → Y` where **Y ⊆ X**	`X → Y` where **Y ⊄ X**
Always holds, so it carries no information	Gives real information about the data
Example: `{Roll, Name} → Name`	Example: `Roll → Name`

- *Completely non-trivial:* `X ∩ Y = ∅`.

(c) Armstrong's Axioms (sound and complete inference rules for FDs)

- **Reflexivity:** if Y ⊆ X, then X → Y.
- **Augmentation:** if X → Y, then XZ → YZ.
- **Transitivity:** if X → Y and Y → Z, then X → Z.
- *Derived rules:* **Union** (X → Y, X → Z gives X → YZ), **Decomposition** (X → YZ gives X → Y and X → Z), **Pseudo-transitivity** (X → Y, WY → Z gives WX → Z).

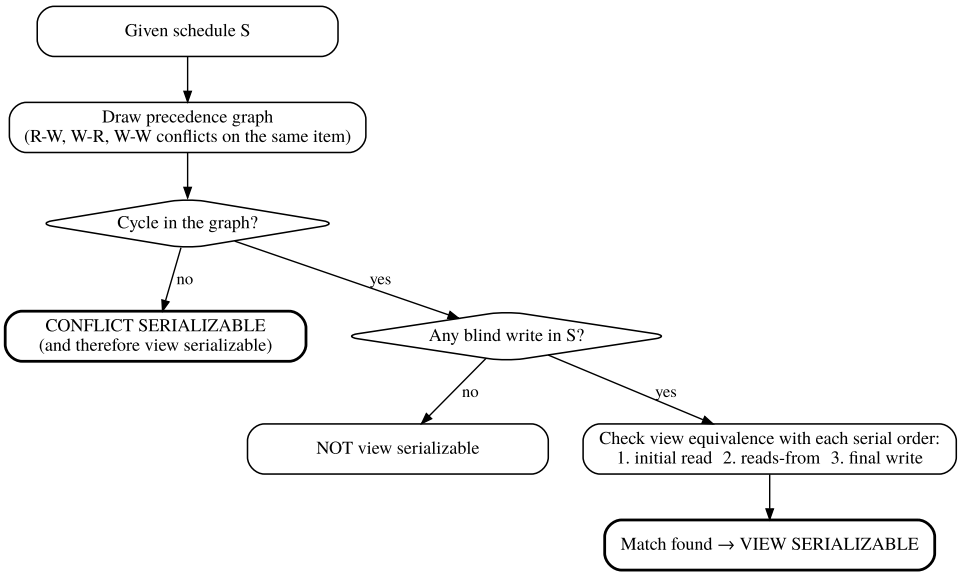
Unit 4: Transactions & Concurrency Control

Q1. Differentiate between conflict and view serializability, and test a given schedule for view serializability. (7 Marks)

1. Introduction / Definition

A schedule is **serializable** if its effect is equivalent to some **serial schedule**. **Conflict serializability** requires that the schedule can be turned into a serial one by **swapping non-conflicting operations**. **View serializability** only requires that the schedule gives every transaction the **same "view"** (the same reads and the same final writes) as a serial schedule.

2. Flowchart - Testing a Schedule



Testing a schedule for conflict and view serializability

3. Core Explanation / Working

(a) Conflict Serializability

- Two operations **conflict** if they belong to **different transactions**, act on the **same data item**, and **at least one is a write**: R-W, W-R or W-W.
- **Test:** build a **precedence graph**. Draw an edge Ti → Tj if an operation of Ti conflicts with, and comes before, an operation of Tj.
- If the graph is **acyclic**, the schedule is conflict serializable. A topological sort gives the equivalent serial order.

(b) View Serializability

- Schedules S and S' are **view equivalent** if, for each data item Q:
    1. **Initial read:** if Ti reads the initial value of Q in S, it also reads the initial value in S'.
    2. **Updated read (reads-from):** if Ti reads a value of Q written by Tj in S, it does so in S' as well.
    3. **Final write:** the transaction that performs the last write on Q in S also does so in S'.
- S is **view serializable** if it is view equivalent to some serial schedule.
- Every conflict-serializable schedule is view serializable, but **not the other way round**. The extra view-serializable schedules always contain **blind writes** (writes without a preceding read).
- Testing view serializability is **NP-complete**.

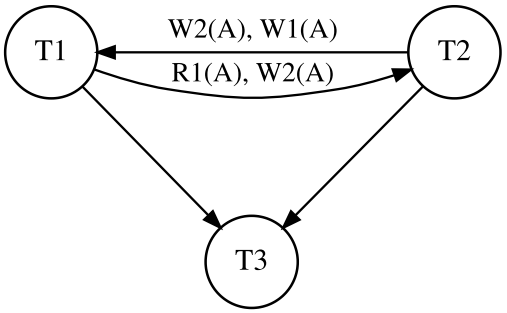
(c) Solved Example - Test for View Serializability

Schedule S

Time	T1	T2	T3
1	R(A)		
2		W(A)	
3	W(A)		
4			W(A)

Step 1: Conflict test with a precedence graph

- R1(A) before W2(A) gives T1 → T2.
- W2(A) before W1(A) gives T2 → T1.
- W1(A) before W3(A) gives T1 → T3. W2(A) before W3(A) gives T2 → T3.



Precedence graph: the cycle T1 → T2 → T1 means S is NOT conflict serializable

Step 2: View test. W2(A) and W3(A) are **blind writes**, so check against the serial order T1 → T2 → T3.

Condition	Schedule S	Serial T1 → T2 → T3	Match?
Initial read of A	T1	T1	Yes
Updated (W-R) reads	None	None	Yes
Final write of A	T3	T3	Yes

- **Result:** S is view equivalent to T1 → T2 → T3, so S is **VIEW SERIALIZABLE** but **not conflict serializable**.

4. Key Characteristics - Conflict vs. View Serializability

Basis	Conflict Serializability	View Serializability
Based on	Order of conflicting operations	Initial read, reads-from, final write
Test	Precedence graph (acyclic)	View equivalence with serial schedules
Complexity	Polynomial (easy)	**NP-complete** (hard)
Blind writes	Not considered specially	Allow extra schedules with blind writes
Relationship	Subset (stricter)	Superset (more permissive)
Used in practice	Yes (2PL, timestamps)	Mostly theoretical

5. Advantages & Disadvantages

Advantages

- Both guarantee **database consistency** while still allowing **concurrent execution**.
- The conflict test is **efficient** and is the basis of practical protocols like 2PL.
- View serializability **accepts more schedules**, which allows more concurrency in theory.

Disadvantages

- Testing view serializability is **NP-complete**, so it is not practical for a real DBMS.
- Conflict serializability **rejects some correct schedules** (those with blind writes).

6. Conclusion / Application

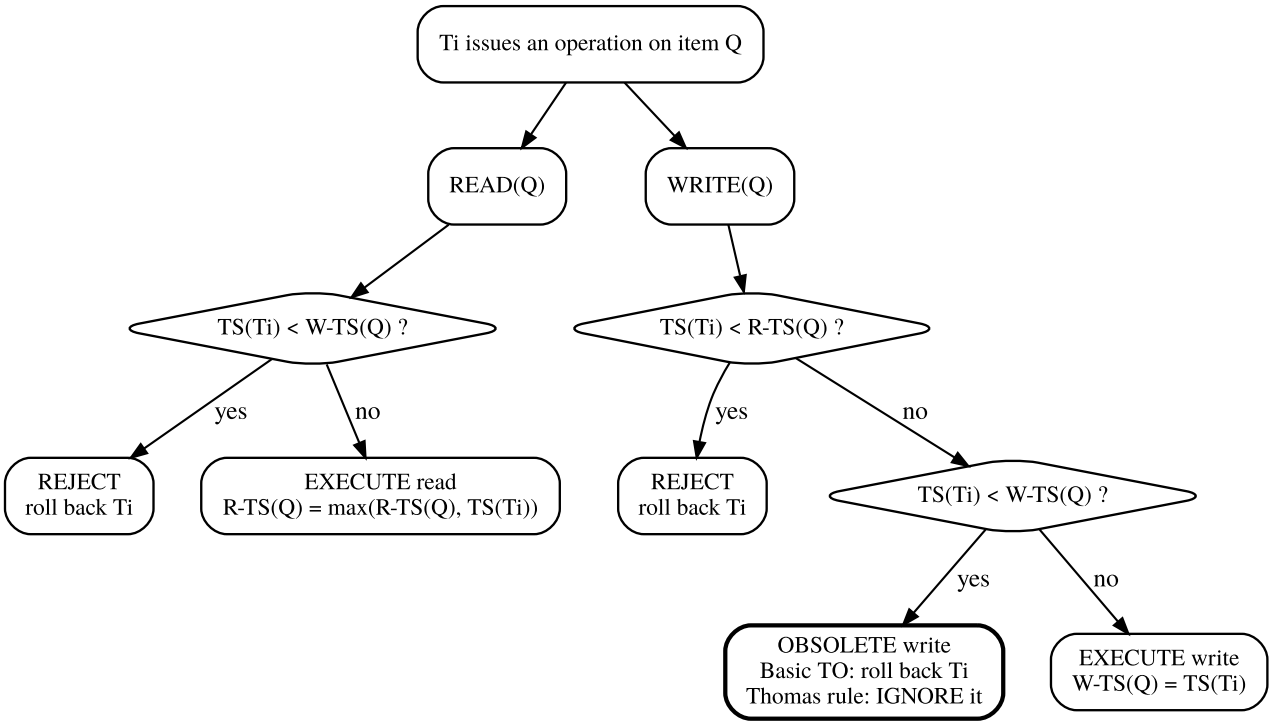
Serializability theory is the **correctness criterion** behind every concurrency-control protocol used in RDBMSs like Oracle and PostgreSQL. Conflict serializability is the form actually enforced.

Q2. Explain the Timestamp Ordering Protocol and how the Thomas Write Rule handles obsolete write operations. (7 Marks)

1. Introduction / Definition

The **Timestamp Ordering (TO) Protocol** is a **non-lock-based** concurrency control method. It gives each transaction a unique **timestamp TS(Ti)** when it starts, and makes sure conflicting operations run in **timestamp order**, so the schedule is equivalent to the serial order of the timestamps. The **Thomas Write Rule** is a modification that **ignores obsolete writes** instead of rolling the transaction back.

2. Flowchart - Basic TO Protocol with Thomas Write Rule



Basic timestamp ordering, with the Thomas Write Rule change

3. Core Explanation / Working

(a) Timestamps

- **TS(Ti):** a unique timestamp given when Ti starts (from the system clock or a logical counter). If Ti starts before Tj, then TS(Ti) < TS(Tj).
- **W-timestamp(Q):** the largest timestamp of any transaction that **wrote** Q successfully.
- **R-timestamp(Q):** the largest timestamp of any transaction that **read** Q successfully.

(b) Rules of the Basic Timestamp Ordering Protocol

- **When Ti issues READ(Q):**
  - If `TS(Ti) < W-TS(Q)`, Ti wants a value that has already been overwritten by a younger transaction. **Reject and roll back Ti.**
  - Otherwise, **execute** the read and set `R-TS(Q) = max(R-TS(Q), TS(Ti))`.
- **When Ti issues WRITE(Q):**
  - If `TS(Ti) < R-TS(Q)`, a younger transaction has already read the old value. **Reject and roll back Ti.**
  - If `TS(Ti) < W-TS(Q)`, Ti is trying to write an **obsolete** value. **Reject and roll back Ti.**
  - Otherwise, **execute** the write and set `W-TS(Q) = TS(Ti)`.
- A rolled-back transaction restarts with a **new, larger timestamp**.

(c) Thomas Write Rule

- It changes **only the second write check**.
- If `TS(Ti) < W-TS(Q)`, the write is **obsolete**, because a younger transaction has already written Q and nobody will ever read Ti's value. So the write is simply **ignored** and Ti continues, with no rollback.

- This allows some **view-serializable schedules that are not conflict serializable**, which increases concurrency.

(d) Solved Example

TS(T1) = 10, TS(T2) = 20, and initially R-TS(Q) = W-TS(Q) = 0.

Step	Operation	Check	Basic TO	Thomas Write Rule
1	R1(Q)	10 ≥ W-TS = 0	Execute, R-TS(Q) = 10	Execute, R-TS(Q) = 10
2	W2(Q)	20 ≥ R-TS = 10, 20 ≥ W-TS = 0	Execute, W-TS(Q) = 20	Execute, W-TS(Q) = 20
3	W1(Q)	10 ≥ R-TS = 10, but 10 < W-TS = 20	**Rollback T1**	**Ignore W1(Q)**, T1 continues

4. Key Characteristics - Basic TO vs. Thomas Write Rule

Basis	Basic Timestamp Ordering	Thomas Write Rule
Obsolete write (TS(Ti) < W-TS(Q))	Rollback Ti	Ignore the write
Schedules allowed	Conflict serializable only	Also some view serializable
Concurrency	Lower	Higher
Number of rollbacks	More	Fewer
Deadlock	Free (no locks)	Free (no locks)

5. Advantages & Disadvantages

Advantages

- It is **deadlock-free**, because no transaction ever waits for a lock.
- It guarantees **serializability** in timestamp order.
- The Thomas Write Rule **reduces unnecessary rollbacks**, which improves throughput.

Disadvantages

- **Starvation** is possible, because a long transaction may be rolled back again and again.
- The schedules may be **non-recoverable** or cause **cascading rollbacks** unless strict TO is used.
- Storing R-TS and W-TS for every data item and updating them on each access adds **overhead**.

6. Conclusion / Application

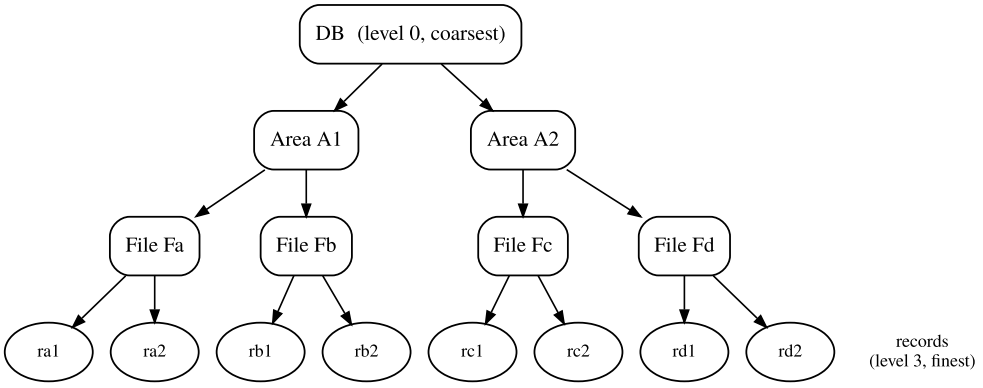
Timestamp-based concurrency control is the basis of **MVCC (Multi-Version Concurrency Control)** used in PostgreSQL, Oracle and distributed databases like Google Spanner.

Q3. Explain multiple granularity locking, lock escalation and the purpose of intention lock modes. (7 Marks)

1. Introduction / Definition

Multiple granularity locking (MGL) lets a transaction lock data items of **different sizes** (database, area, file, record), arranged as a **tree hierarchy**. A lock on a node **implicitly locks all its descendants**. **Intention locks** (IS, IX, SIX) are placed on ancestor nodes to show that **finer-level locks are held** below them.

2. Block Diagram - Granularity Hierarchy Tree



Granularity hierarchy: lock top-down, unlock bottom-up

3. Core Explanation / Working

(a) Lock Modes

- **S (Shared):** read access to the node and all its descendants.
- **X (Exclusive):** read/write access to the node and all its descendants.
- **IS (Intention-Shared):** explicit S locks will be taken at a **lower level**.
- **IX (Intention-Exclusive):** explicit X or S locks will be taken at a **lower level**.
- **SIX (Shared + Intention-Exclusive):** the whole subtree is locked in S mode, and some lower nodes will get X locks. Example: read a whole table and update a few rows.

(b) Compatibility Matrix (Y = compatible, N = conflict)

Requested ↓ / Held →	IS	IX	S	SIX	X
IS	Yes	Yes	Yes	Yes	No
IX	Yes	Yes	No	No	No
S	Yes	No	Yes	No	No
SIX	Yes	No	No	No	No
X	No	No	No	No	No

(c) Rules of the Multiple Granularity Locking Protocol (for transaction Ti)

1. Ti must follow the **compatibility matrix**.
2. The **root** must be locked first, in any mode.
3. Ti can lock node Q in **S or IS** mode only if it holds the **parent** in **IX or IS** mode.
4. Ti can lock node Q in **X, SIX or IX** mode only if it holds the **parent** in **IX or SIX** mode.
5. Ti follows **two-phase locking**: it cannot lock any node after it has unlocked one.
6. Ti can **unlock** Q only if it holds **no locks on any child** of Q.

Example: T1 wants to update record `ra1`. It takes **IX** on DB, **IX** on A1, **IX** on Fa and then **X** on ra1. At the same time, T2 can read `rb1` by taking IS on DB, IS on A1, IS on Fb and S on rb1. Both run in parallel because IX and IS are compatible.

(d) Purpose of Intention Locks

- Without them, to lock a whole file the DBMS would have to **search every descendant** to check for conflicting locks, which is very slow.
- With intention locks, the DBMS only checks the **node and its ancestors**. If any ancestor shows a conflicting intention, the request waits.
- This lets **coarse and fine locks coexist** safely and efficiently.

(e) Lock Escalation

- When a transaction holds **too many fine-grained locks** (for example more than 5000 row locks on one table), the DBMS **replaces them with one coarser lock** (a table lock).
- **Why:** it reduces **memory use** for the lock table and the **overhead of lock management**.
- **Cost:** concurrency drops, because other transactions are now blocked from the whole table.
- Used in **SQL Server and DB2** (escalation from row to page to table).

4. Key Characteristics - Fine vs. Coarse Granularity

Basis	Fine Granularity (Record)	Coarse Granularity (Table/DB)
Concurrency	**High**	Low
Locking overhead	High (many locks)	**Low** (few locks)
Lock table memory	Large	Small
Best for	Short transactions on few rows	Long transactions scanning whole tables
Deadlock chance	Higher	Lower

5. Advantages & Disadvantages

Advantages

- It **balances concurrency and overhead** by choosing the right lock size for each transaction.
- Intention locks make conflict detection **fast**, since there is no need to scan the subtree.
- It ensures **serializability**, because it is based on two-phase locking.

Disadvantages

- It is **complex to implement**, with five lock modes and hierarchical rules.
- **Deadlocks are still possible**, because the protocol is lock-based.
- Lock escalation can suddenly **reduce concurrency** and cause blocking.

6. Conclusion / Application

Multiple granularity locking with intention locks is implemented in **Microsoft SQL Server, IBM DB2 and MySQL InnoDB** (IS/IX table-level intention locks with row locks).

Q4. Short Definitions (2 Marks Each)

(a) Dirty Read Problem (Temporary Update / W-R Conflict)

- Happens when a transaction **reads data written by another transaction that has not yet committed**. If that transaction then **aborts**, the first transaction has used an invalid value.
- Example: T1 writes A = 500 (uncommitted), T2 reads A = 500, then T1 rolls back, so T2 holds a "dirty" value.

(b) Unrepeatable Read Problem (R-W Conflict)

- Happens when a transaction **reads the same item twice and gets different values**, because another transaction **modified and committed** it between the two reads.
- Example: T1 reads A = 100, T2 updates A = 200 and commits, then T1 reads A again and gets 200.

(c) Starvation vs. Deadlock

Starvation	Deadlock
A transaction **waits indefinitely** while others keep proceeding	Two or more transactions **wait for each other in a cycle**, so none can proceed
Caused by an unfair scheduling or victim-selection policy	Caused by circular wait on locks
Other transactions are progressing	The involved transactions are all blocked
Fix: aging, FIFO lock queue	Fix: prevention (wait-die, wound-wait), detection (wait-for graph) and recovery

---

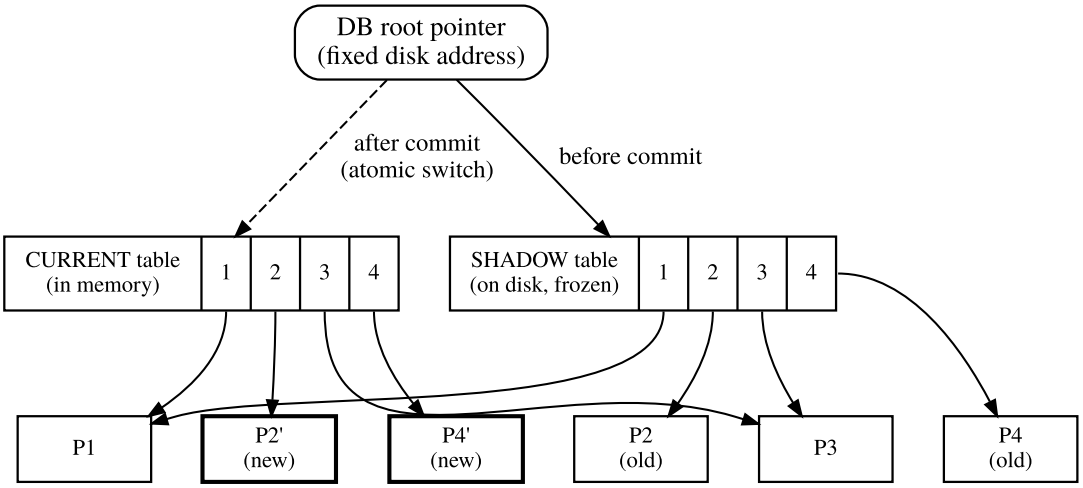
Unit 5: Recovery & File Organization

Q1. Explain the shadow paging database recovery technique along with its primary advantages and disadvantages. (7 Marks)

1. Introduction / Definition

Shadow paging is a recovery technique that **does not need a log** (in a single-user environment). The database is divided into fixed-size **pages**, and two page tables are kept during a transaction: the **current page table** and the **shadow page table**. The shadow table is never changed during the transaction, so it always points to the last consistent state.

2. Block Diagram



Shadow paging: the shadow table still points to the old pages; pages 2 and 4 were copied on write

3. Core Explanation / Working

- **Step 1 - Start of transaction:** the current page table is **copied** to make the shadow page table, which is saved to stable storage. Both point to the same disk pages.
- **Step 2 - Read operation:** pages are found through the **current page table**.
- **Step 3 - Write operation (copy-on-write):**
  - Find a **free page** on disk.
  - Copy the old page into the free page and apply the update to this **new copy**.
  - Change the **current page table** entry to point to the new page.
  - The shadow page table still points to the **old, unchanged** page.
- **Step 4 - Commit:**
  1. Write all modified data pages to disk.
  2. Write the current page table to disk.
  3. **Atomically** change the fixed **DB root pointer** from the shadow table to the current table. This single disk write is the commit point.
- **Step 5 - Abort / crash recovery:**
  - Discard the current page table and the new pages.
  - The root pointer still points to the **shadow table**, which reflects the consistent state from before the transaction.
  - **No UNDO and no REDO** is needed. Free the unused pages (garbage collection).

4. Key Characteristics - Shadow Paging vs. Log-Based Recovery

Basis	Shadow Paging	Log-Based Recovery
Log required	No (single user)	Yes
Recovery operations	None (NO-UNDO / NO-REDO)	UNDO and/or REDO
Recovery speed	Very fast	Slower (log scanning)
Commit overhead	High (copy page table, flush pages)	Low (flush log records)
Data locality	Destroyed (pages scattered)	Preserved
Concurrency support	Difficult	Easy

5. Advantages & Disadvantages

Advantages

- **No log overhead** in a single-user environment.
- **Very fast recovery**, because no undo or redo is needed. Reverting to the shadow table is enough.
- Simple **atomic commit** using a single pointer switch.

Disadvantages

- **Commit overhead:** the entire page table must be written at commit, which is costly for large databases.
- **Data fragmentation:** updated pages move to new locations, so related pages are no longer **contiguous**.
- **Garbage collection** is needed to reclaim old pages after commit.
- Hard to extend to **concurrent transactions**, where logs and checkpoints are still needed.

6. Conclusion / Application

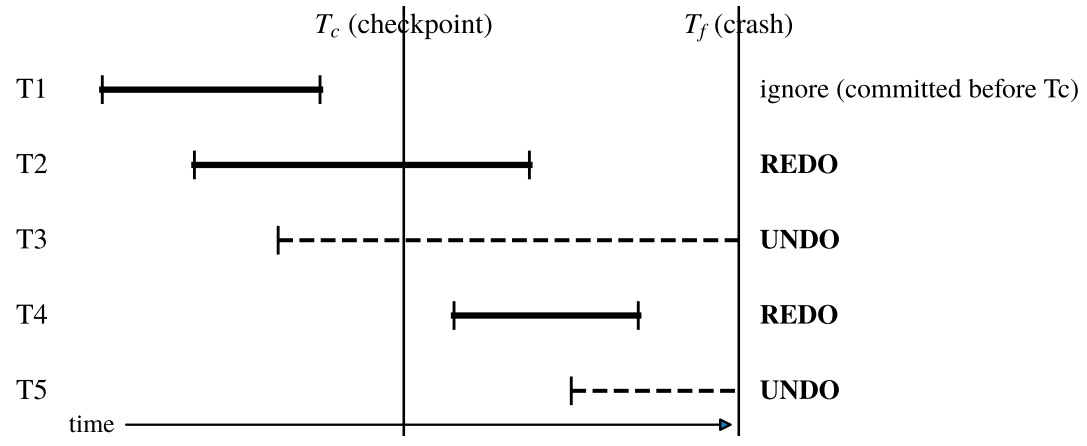
Shadow paging is used in **System R (early IBM RDBMS)**, the **SQLite rollback journal concept**, and modern **copy-on-write file systems** like ZFS and Btrfs.

Q2. Explain Checkpointing. Differentiate between deferred and immediate database modification with log records. (7 Marks)

1. Introduction / Definition

Log-based recovery keeps a **log** (a sequential file in stable storage) of every database modification, so the system can **UNDO** or **REDO** transactions after a failure. A **checkpoint** is a point at which the DBMS writes all buffers to disk and records `<checkpoint>` in the log. This limits how much of the log must be scanned during recovery.

2. Block Diagram - Checkpoint Recovery Timeline



Checkpoint recovery: transactions are classified by when they started and committed relative to the checkpoint Tc and the crash Tf

3. Core Explanation / Working

(a) Log Record Types

- `<Ti start>`: transaction Ti has started.
- `<Ti, X, V_old, V_new>`: Ti changed item X from V_old to V_new.
- `<Ti commit>`: Ti has committed.

- `<Ti abort>`: Ti has aborted.

(b) Deferred Database Modification (NO-UNDO / REDO)

- All writes are **postponed until the transaction commits**. They are recorded only in the log and a local buffer.
- Log records hold **only the new value**: `<Ti, X, V_new>`.
- After `<Ti commit>` is written to the log, the updates are applied to the database.
- **Recovery:** REDO(Ti) if the log has both `<Ti start>` and `<Ti commit>`. Otherwise ignore Ti. **UNDO is never needed.**

(c) Immediate Database Modification (UNDO / REDO)

- Writes may be applied to the database **before the transaction commits**, as long as the log record is written first (the WAL rule).
- Log records hold **both old and new values**: `<Ti, X, V_old, V_new>`.
- **Recovery:**
  - **UNDO(Ti)** if the log has `<Ti start>` but no `<Ti commit>`. Restore the old values, scanning **backward**.
  - **REDO(Ti)** if the log has both `<Ti start>` and `<Ti commit>`. Reapply the new values, scanning **forward**.

(d) Example - T0: A = 1000 to 950, B = 2000 to 2050. T1: C = 700 to 600.

Deferred modification log	Immediate modification log
⟨T0 start⟩	⟨T0 start⟩
⟨T0, A, 950⟩	⟨T0, A, 1000, 950⟩
⟨T0, B, 2050⟩	⟨T0, B, 2000, 2050⟩
⟨T0 commit⟩	⟨T0 commit⟩
⟨T1 start⟩	⟨T1 start⟩
⟨T1, C, 600⟩	⟨T1, C, 700, 600⟩
crash	**crash**
Recovery: **REDO(T0)**; ignore T1	Recovery: **REDO(T0)**; **UNDO(T1)**, so C = 700

(e) Checkpointing

- **Problem without checkpoints:** the whole log must be scanned, and many already-saved transactions get redone needlessly.
- **Steps to take a checkpoint:**
  1. Write all log records in main memory to **stable storage**.
  2. Write all **modified buffer blocks** to disk.
  3. Write a `<checkpoint L>` record to the log, where L is the list of active transactions.
- **Recovery using a checkpoint:**
  - Scan backward to the last `<checkpoint L>`.
  - Only transactions in L, or those that started after the checkpoint, are considered.
  - Committed ones go into the **redo-list**. Uncommitted ones go into the **undo-list**.
  - Transactions that committed **before** the checkpoint are **ignored** (T1 in the diagram).

4. Key Characteristics - Deferred vs. Immediate Modification

Basis	Deferred Modification	Immediate Modification
When DB is updated	Only after commit	Before or after commit
Log record format	`<Ti, X, V_new>`	`<Ti, X, V_old, V_new>`
Recovery operations	**REDO only** (NO-UNDO)	**UNDO and REDO**
Old value needed	No	Yes
Buffer requirement	Large (holds all updates)	Small
Concurrency	Lower	Higher
Commit speed	Slower (all writes happen at commit)	Faster

5. Advantages & Disadvantages

Advantages

- Logs make sure the **Atomicity and Durability** properties of ACID survive crashes.
- Checkpoints **reduce recovery time** sharply by limiting the log scan.
- Deferred modification is **simple** (no undo). Immediate modification gives **better performance** and concurrency.

Disadvantages

- Log writing adds **I/O overhead** to every update.
- Taking a checkpoint **briefly pauses** transaction processing (unless fuzzy checkpoints are used).
- Deferred modification needs **large buffers** for long transactions.

6. Conclusion / Application

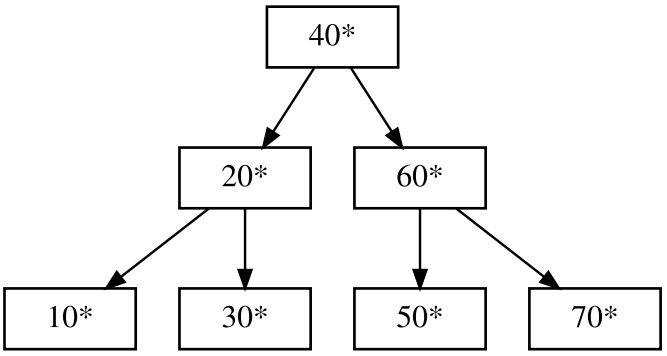
Immediate modification with checkpoints is the basis of **ARIES**, the recovery algorithm used in **IBM DB2, SQL Server and PostgreSQL (WAL)**.

Q3. Compare B-Trees and B+ Trees. Demonstrate step-by-step insertion in a B-Tree of order m. (7 Marks)

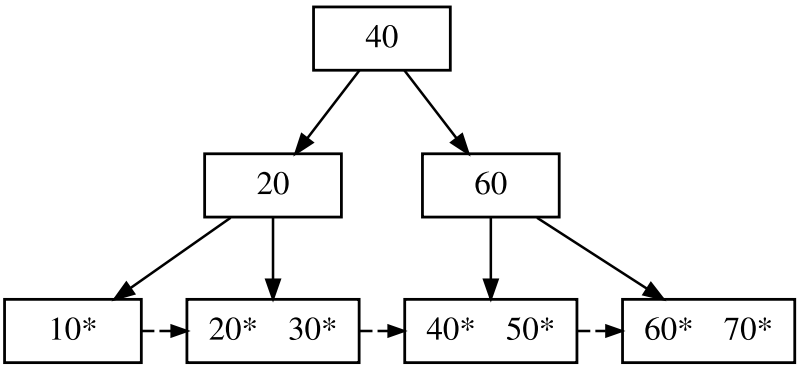
1. Introduction / Definition

A **B-Tree of order m** is a **self-balancing multi-way search tree** in which every node has at most **m children** and **m − 1 keys**, every non-root node has at least **⌈m/2⌉ children**, and **all leaves are at the same level**. A **B+ Tree** is a variant in which **all data records are stored only in leaf nodes**, internal nodes hold only index keys, and the leaves are **linked sequentially**.

2. Block Diagram - Structure Comparison



B-Tree of order 3: every node holds keys AND data pointers (*)



B+ Tree of order 3: data only in the leaves, and the leaves are linked in order

3. Core Explanation / Working

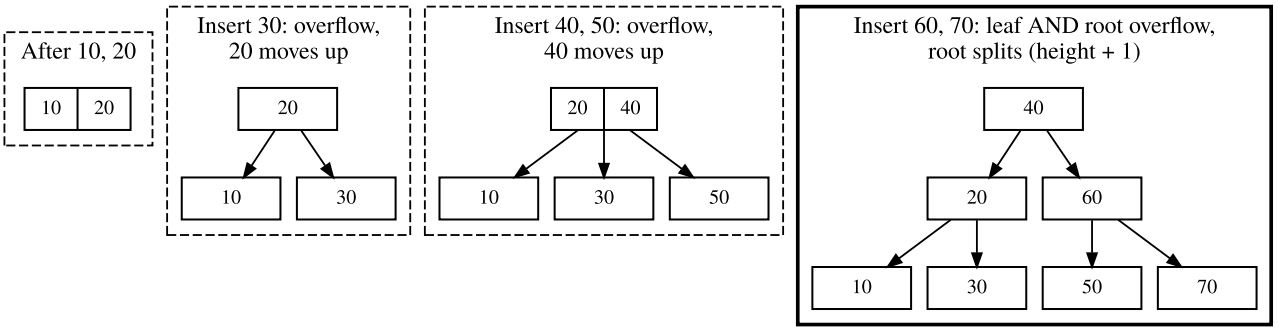
(a) Properties of a B-Tree of order m

- Every node has at most **m children** and **m − 1 keys**.
- Every internal node (except the root) has at least **⌈m/2⌉ children**, that is **⌈m/2⌉ − 1 keys**.
- The root has at least **2 children** (if it is not a leaf).
- All leaves are at the **same depth**, so the tree is perfectly balanced.
- The keys in each node are kept in **sorted order**.

(b) Insertion Algorithm

1. Search for the correct **leaf** node for the new key.
2. If the leaf has fewer than m − 1 keys, **insert in sorted order**. Done.
3. If the leaf **overflows** (it now has m keys), **split** it:
    - The **median key moves up** to the parent.
    - The left half and the right half become two separate nodes.
4. If the parent overflows too, split it recursively. If the **root splits**, a new root is created and the **height increases by 1**.

(c) Solved Example - Insert 10, 20, 30, 40, 50, 60, 70 into a B-Tree of order m = 3 (max 2 keys per node)



B-Tree (order m = 3) insertion of 10, 20, ..., 70: each overflow pushes the median key up

Step	Key inserted	What happens	Tree after the step
1-2	10, 20	Fit in the root	[10 | 20]
3	30	Root overflows [10 | 20 | 30]: 20 moves up	root [20], leaves [10] [30]
4	40	Goes to leaf [30]	[30 | 40]
5	50	Leaf overflows [30 | 40 | 50]: 40 moves up	root [20 | 40], leaves [10] [30] [50]
6	60	Goes to leaf [50]	[50 | 60]
7	70	Leaf overflows: 60 moves up, so the root becomes [20 | 40 | 60] and overflows: 40 moves up	root [40], children [20] [60], leaves [10] [30] [50] [70]

Final tree: height = 3, all leaves at the same level, and every node has between 1 and 2 keys.

4. Key Characteristics - B-Tree vs. B+ Tree

Basis	B-Tree	B+ Tree
Data pointers stored in	All nodes (internal + leaf)	**Leaf nodes only**
Duplicate keys	No, each key appears once	Yes, keys repeat in internal nodes
Leaf linking	Leaves not linked	Leaves form a **linked list**
Search time	Variable (may end at an internal node)	Uniform (always reaches a leaf)
Range / sequential queries	Slow (tree traversal)	**Fast** (follow leaf links)
Fan-out of internal nodes	Lower (data pointers take space)	**Higher**, so the tree is shorter
Deletion	Complex (from internal nodes)	Simpler (always from a leaf)
Usage	Some file systems	**Almost all DBMS indexes**

5. Advantages & Disadvantages

Advantages

- **Balanced height**, so search, insert and delete take **O(log n)** time.
- **Few disk I/Os**, because large nodes match disk block size.
- The B+ Tree supports **efficient range queries and sequential access**.

Disadvantages

- Insertions and deletions need **splits and merges**, which are complex to implement.
- Nodes may be only **half full**, which wastes space.
- The B+ Tree stores keys **twice** (in internal nodes and leaves).

6. Conclusion / Application

B+ Trees are the **default index structure** in MySQL InnoDB, Oracle, PostgreSQL and SQL Server. B-Trees are used in file systems like NTFS and HFS+.

Q4. Short Definitions (2 Marks Each)

(a) Write-Ahead Logging (WAL)

- A rule that says **the log record for a change must be written to stable storage before** the changed data page is written to disk.
- In addition, all log records of a transaction, including `<Ti commit>`, must be on stable storage **before the transaction is declared committed**.
- This makes **UNDO and REDO** possible after a crash. It is used in PostgreSQL, SQLite and others.

(b) Static vs. Dynamic Hashing

Static Hashing	Dynamic Hashing
The **number of buckets is fixed** in advance	Buckets **grow and shrink** as data changes
Overflow is handled by **overflow chains**	Overflow is handled by **splitting buckets**
Performance falls as the database grows	Performance stays stable
Example: `h(K) = K mod 10`	Example: **Extendible hashing**, Linear hashing

(c) Primary vs. Clustering vs. Secondary Index

- **Primary index:** built on the **ordering key field** (primary key) of a **sorted** file. It is **sparse**, with one entry per block. There is only one per file.
- **Clustering index:** built on an **ordering non-key field** (which can repeat) of a sorted file. There is one entry per **distinct value**.
- **Secondary index:** built on a **non-ordering field**. The file is not sorted on this field, so the index must be **dense**. A file can have **many** secondary indexes.
